

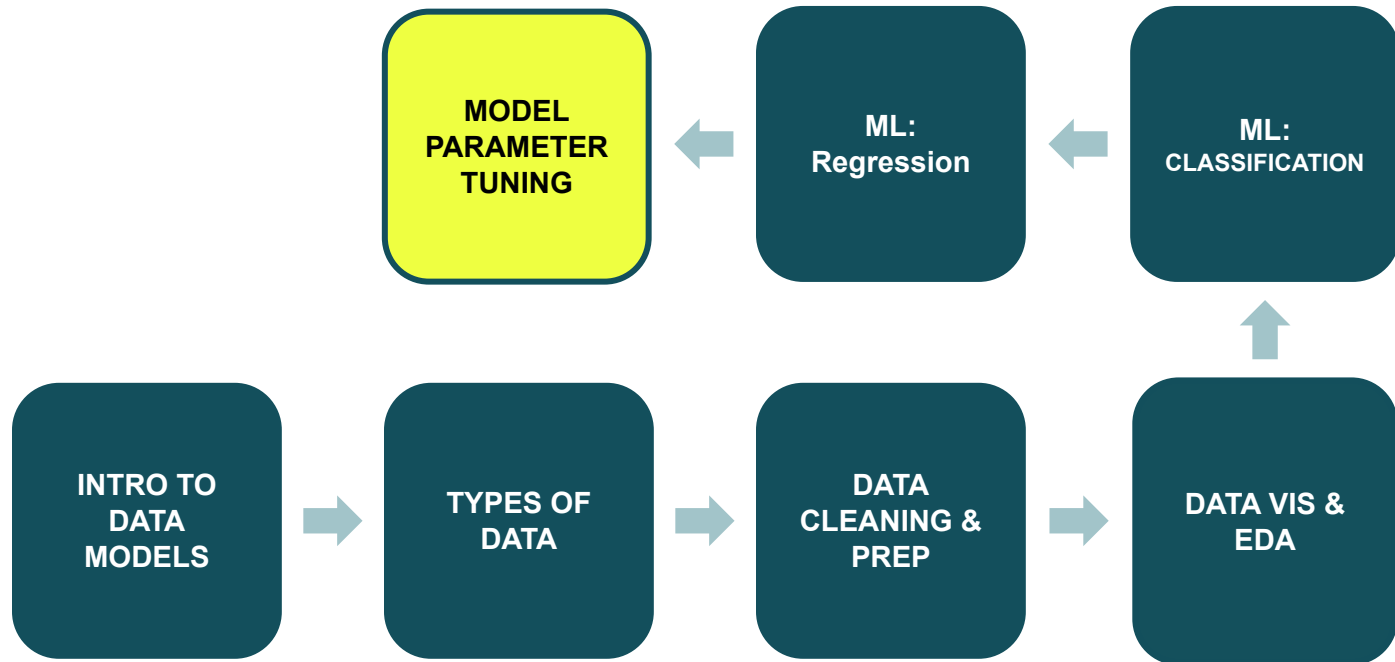
Model Parameter Tuning

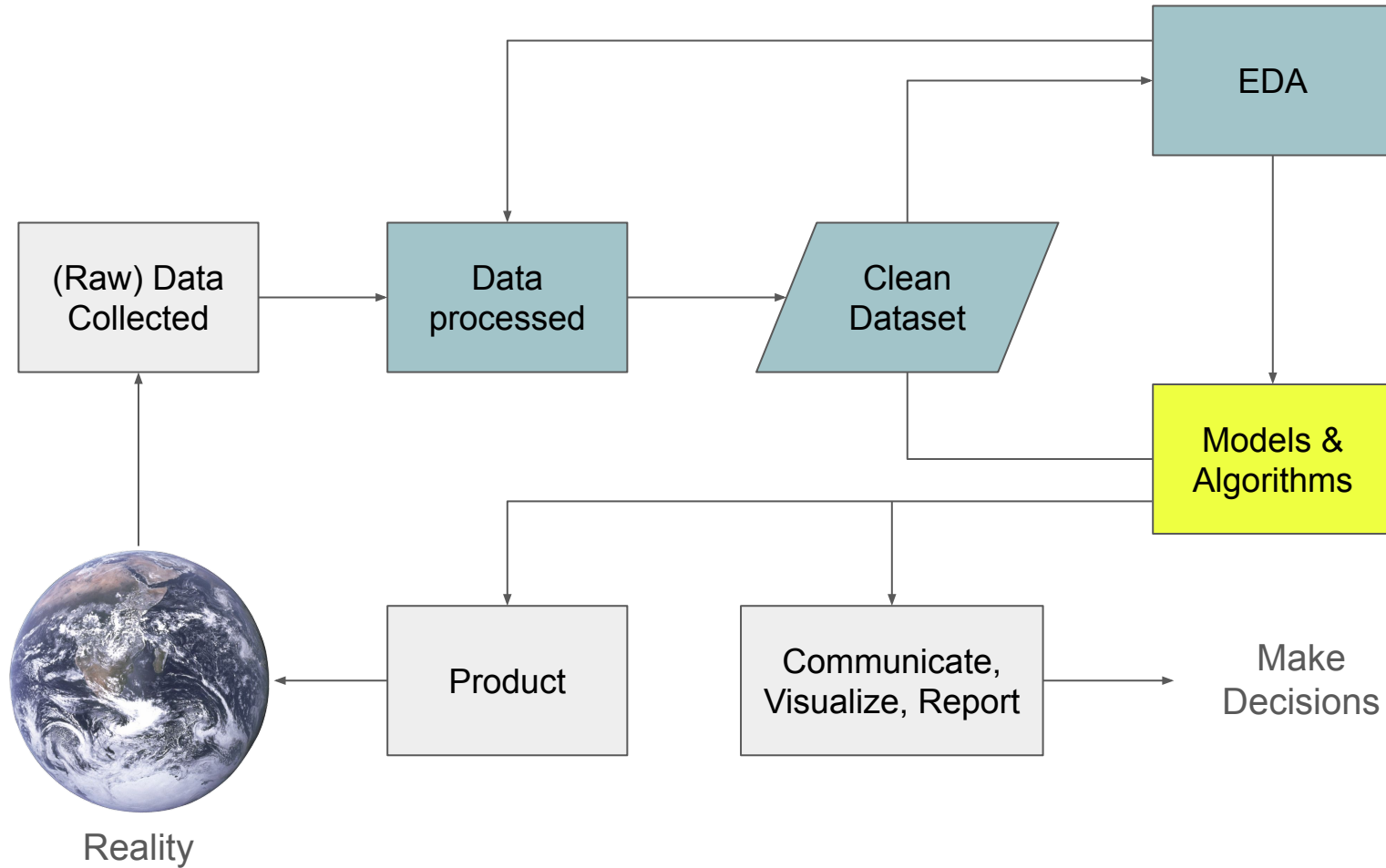
CPE 232: Data Models

Dr. Sansiri Tarnpradab

Department of Computer Engineering, KMUTT

Review

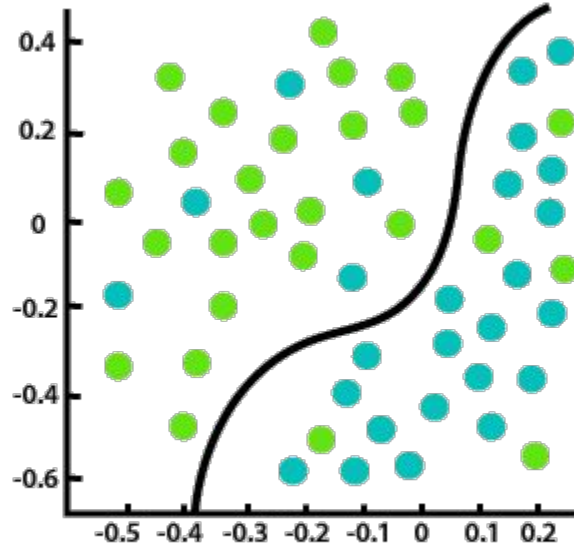




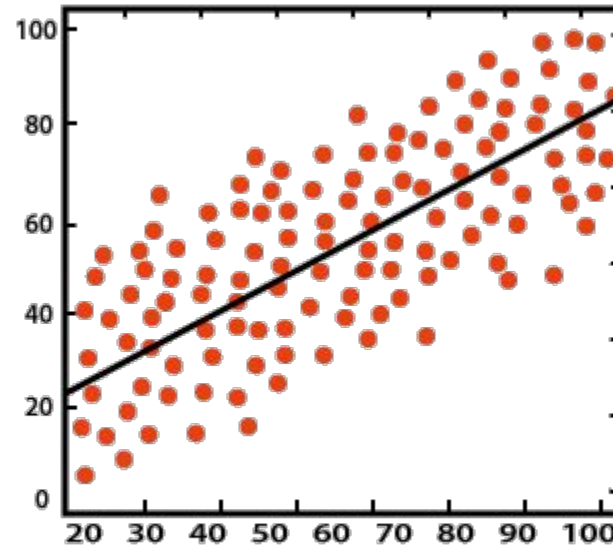
Outline

- Recap (Regression)
- Introducing model parameters
- Learnable parameters vs Hyperparameters
- Examples in ML algorithms
- Necessity & Benefits
- Important Terms
- Methods of hyperparameter tuning
- Challenges
- Evaluation

So far...



Classification



Regression

Ref: <https://www.javatpoint.com/regression-vs-classification-in-machine-learning>

Prediction

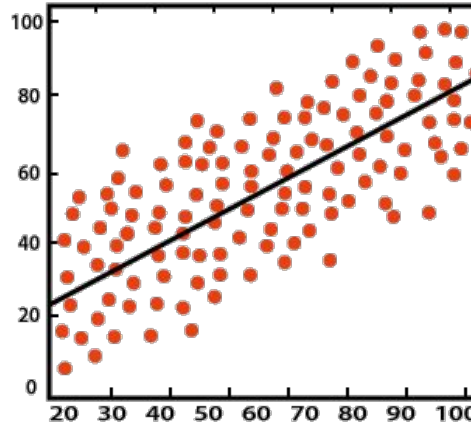
Numerical values

Time-series

Relationship

Independent variables (x)
Aka Predictors

Dependent variables (y)
Aka Outcome



Regression

Evaluation
MSE, RMSE, MAE

Data = Model + Error

Linear Regression

Polynomial Regression
(degree>1)

Decision Tree Regression

Model Parameters

Model Parameter: What is it?

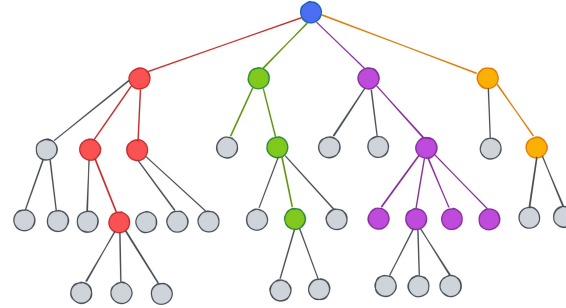
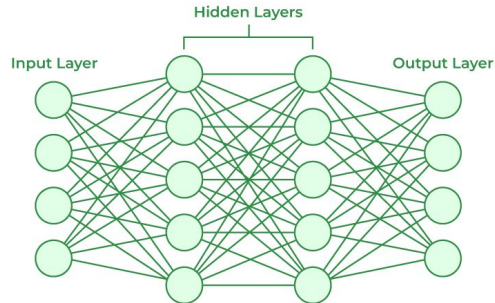
- Coefficients or weights learned by a ML algorithm during the training process
- Define the structure of the model
- Used to make predictions or classify new data points
- Adjustable to minimize the difference between the predicted and actual outcomes

Learnable Parameters	Hyperparameters
• Are optimized during the training process using optimization techniques • Adjusted iteratively to minimize a loss function • Examples: coefficients in linear regression, the weights in neural networks	• Are set before the training process begins • Are set by the user or chosen through techniques like hyperparameter tuning (i.e. not directly learned from the data) • Examples: learning rate in gradient descent, the depth of a decision tree, the number of hidden layers in a neural network.

Examples in ML Algorithms

Neural Networks

Learning rate
Number of layers
Number of input nodes
Number of hidden nodes
Activation function
Batch size

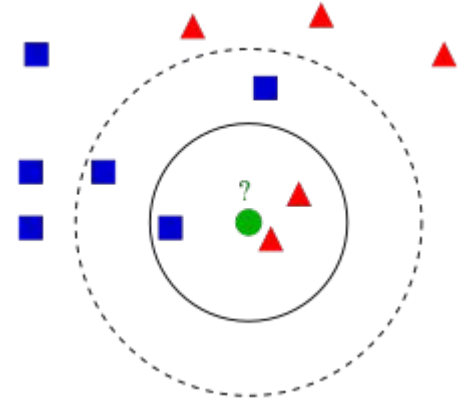


Decision Tree

Max depth of the tree
Min No. of samples split
Min No. of samples leaf
Max No. of features
Criterion
Splitter

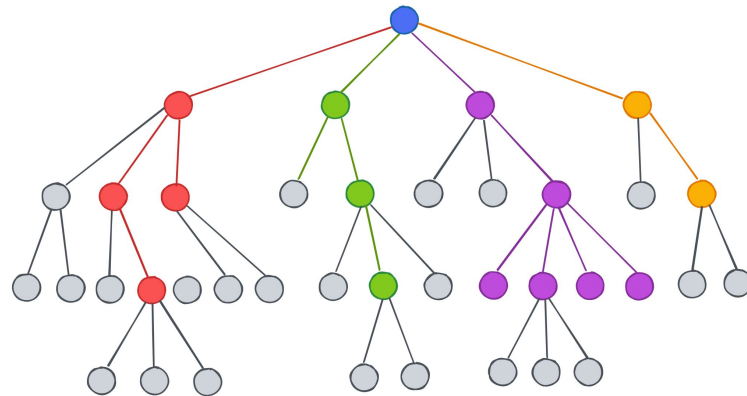
K-Nearest Neighbors

Number of neighbors
Distance metric



Decision Tree Hyperparameters

- **Maximum depth of the tree** (`max_depth`)
- **Minimum Number of samples split** (`min_samples_split`)
 - ◆ If the number of samples at a node is less than this value, the node will not be split further.
- **Minimum Number of samples leaf** (`min_samples_leaf`)
 - ◆ If a split results in a leaf node with fewer samples than `min_samples_leaf`, the split is disregarded.



- **Maximum Number of features** (`max_features`)
 - ◆ Max no. of features to consider when looking for the best split at each node.
- **Criterion**
 - ◆ Entropy or Gini for information gain
- **Splitter**
 - ◆ Random split or Best split

Hyperparameter: Why Tuning?

- **Optimizing Performance**

- Tuning helps in finding the optimal set of hyperparameters
- Result in the best possible performance on unseen data

- **Robustness**

- To make the model more robust to variations in the data or changes in the environment
- More reliable in practical applications

- **Reducing Computational Costs**

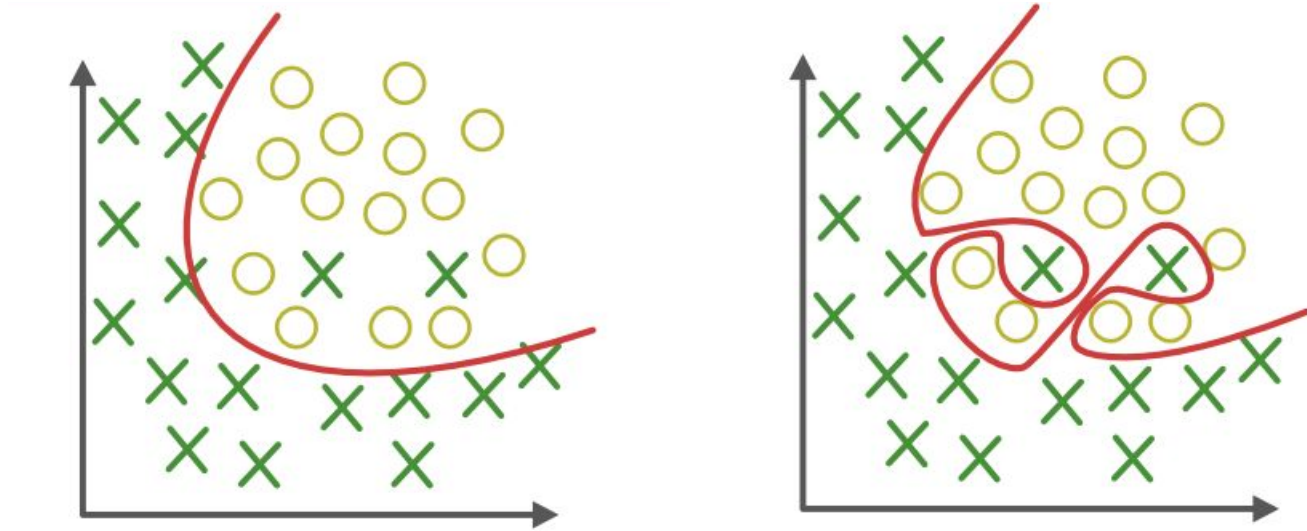
- By finding the most efficient set of hyperparameters.
- Reduce computational time & resources while still achieving good performance.

- **Avoiding Overfitting**

- If hyperparameters are not tuned properly, the model may become too complex
- To find the right balance between model complexity and generalization

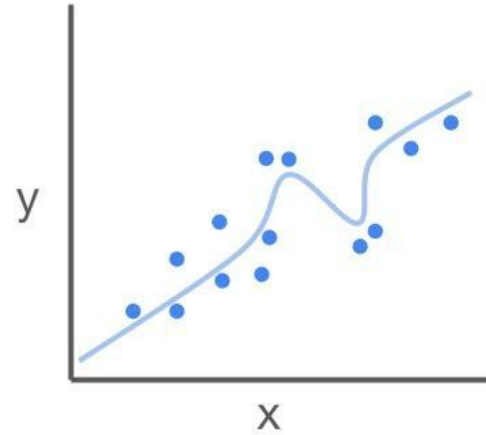
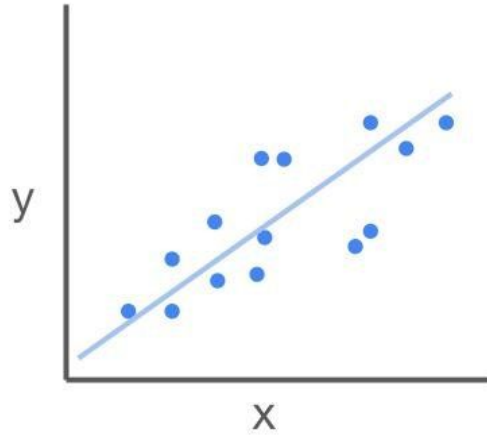
~~Perfection~~ **Overfitting**

Perfection in Classification?



Ref: <https://www.geeksforgeeks.org/underfitting-and-overfitting-in-machine-learning/>

Perfection in Regression?



Ref: <https://crunchingthedata.com/regression-overfitting/>



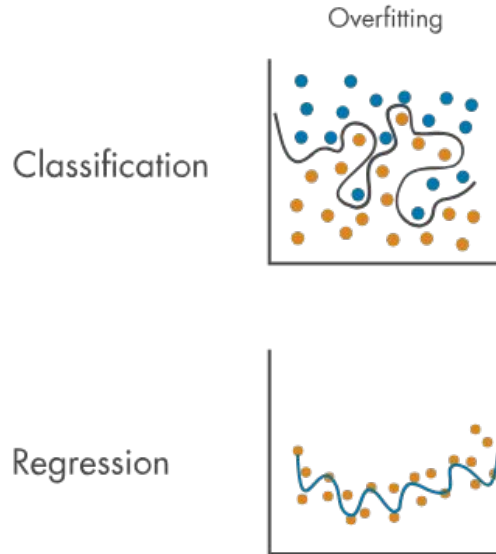
OVERKILL

When you have come-alongs that just need to be used

Ref: <https://www.pinterest.com/pin/280700989254721828/>

Overfitting

- Occurs when a machine learning model learns the training data too well.
- The model captures noise and random fluctuations instead of the pattern.



Characteristics:

- High training accuracy, poor test accuracy
- Model is too complex
 - Too many parameters
 - Deep trees for Decision Tree
- Fails to generalize to new, unseen data

Overfitting: Prevention

- Simplify the Model
 - Reduce the number of parameters or prune decision trees.
- Regularization
 - Use L1/L2 regularization to penalize large weights
 - Examples: Logistic regression or Neural networks.
- More Training Data
 - Helps the model learn general patterns rather than noise.
- Cross-Validation
- Dropout
 - For Neural Networks, randomly drop connections between neurons
 - Purpose: To prevent reliance on specific paths.

Cross-Validation

A technique for evaluating machine learning models by splitting the data into multiple training and validation sets to ensure the model generalizes well.

Why so?

- Ensure the model works well on different subsets of data
- More reliable estimate of model performance compared to a single train-test split

Types of Cross-Validation

- **k-Fold Cross-Validation**

- Data is split into k folds, and the process is repeated k times.

- **Stratified k-Fold Cross-Validation**

- Ensures each fold has approximately the same class distribution as the original dataset
- Imbalanced dataset

- **Leave-One-Out Cross-Validation (LOO-CV)**

- Uses one sample as the test set and the rest as training data
- Computationally expensive but gives an unbiased estimate

- **Time Series Cross-Validation**

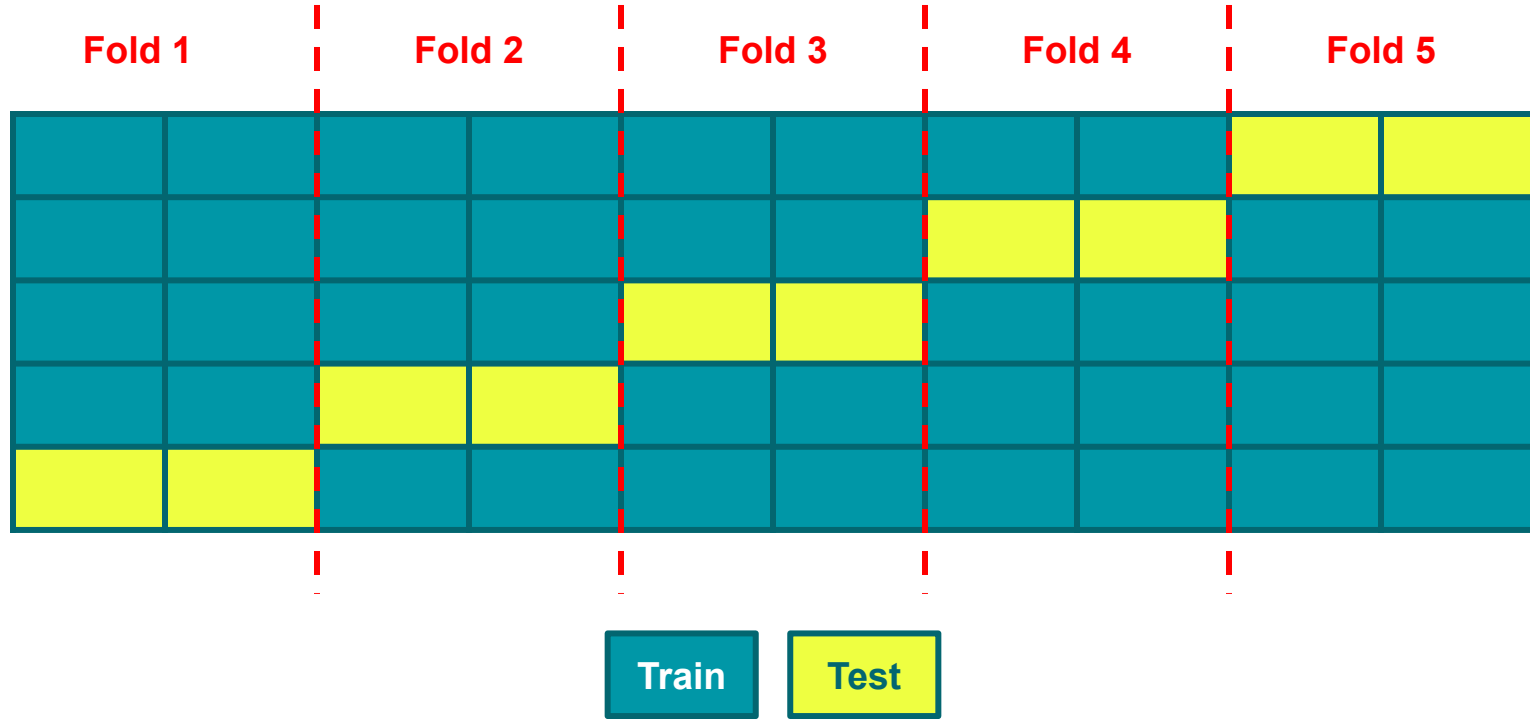
- Used for time-dependent data where past observations should not "see" future data
- Training is done on past data, and testing is done on future data



k-Fold Cross-Validation



5-Fold Cross-Validation



k-Fold Cross-Validation

Let's say...

- Dataset of 10 samples:
1, 2, 3, 4, 5, 6, 7, 8, 9, 10
- Labeled as Class 😊 & Class 😈
- 5-Fold Cross-Validation

What happens:

- The dataset is randomly split into 5 folds
- Each fold becomes the test set once
- The remaining folds are used for training

Round	Train	Test
1	3, 4, 5, 6, 7, 8, 9, 10	1, 2
2	1, 2, 5, 6, 7, 8, 9, 10	3, 4
3	1, 2, 3, 4, 7, 8, 9, 10	5, 6
4	1, 2, 3, 4, 5, 6, 9, 10	7, 8
5	1, 2, 3, 4, 5, 6, 7, 8	9, 10

If the data is imbalanced (e.g., 90% Class A, 10% Class B), some folds may have very few or no instances of Class B, thus leading to biased training.

Example #1: Balanced Data

Dataset of 10 samples:

1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Corresponding Class: 😊 or 😈

- 1 → 😊
- 2 → 😈
- 3 → 😊
- 4 → 😈
- 5 → 😊
- 6 → 😈
- 7 → 😊
- 8 → 😈
- 9 → 😊
- 10 → 😈

Round	Train	Test
1	😊😈😊😈😊😈😊😈	😊😈
2	😊😈😊😈😊😈😊😈😊😈	😊😈
3	😊😈😊😈😊😈😊😈😊😈	😊😈
4	😊😈😊😈😊😈😊😈😊😈	😊😈
5	😊😈😊😈😊😈😊😈😊😈	😊😈

Example #2: Imbalanced Data

Dataset of 10 samples:

1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Corresponding Class: 😇 or 😈

- 1 → 😇
- 2 → 😇
- 3 → 😇
- 4 → 😇
- 5 → 😈
- 6 → 😈
- 7 → 😇
- 8 → 😇
- 9 → 😇
- 10 → 😇

Round	Train	Test
1	😇😇😈😈😇😇😇😇	😇😇
2	😇😇😈😈😇😇😇😇	😇😇
3	😇😇😇😇😇😇😇😇	😈😈
4	😇😇😇😇😈😈😇😇	😇😇
5	😇😇😇😇😈😈😇😇	😇😇

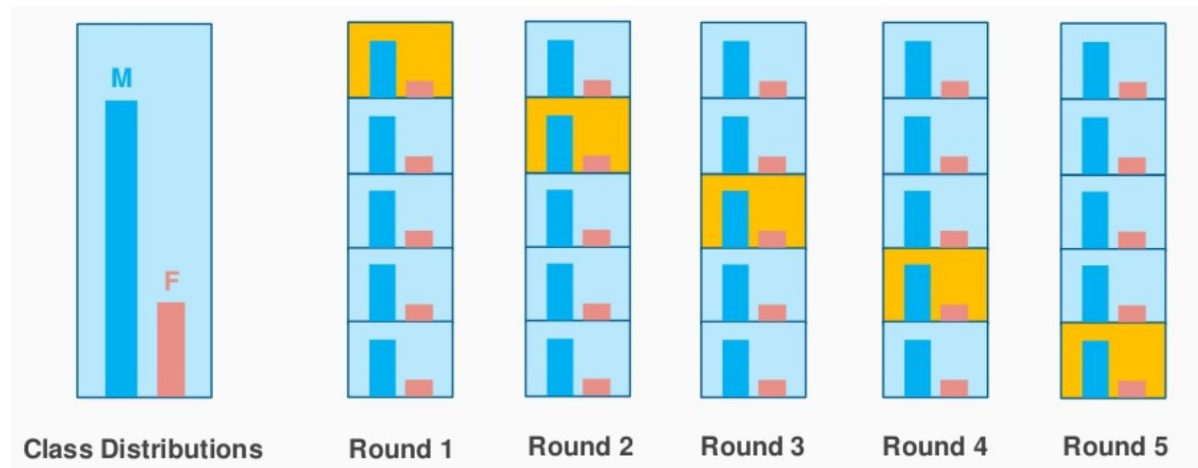
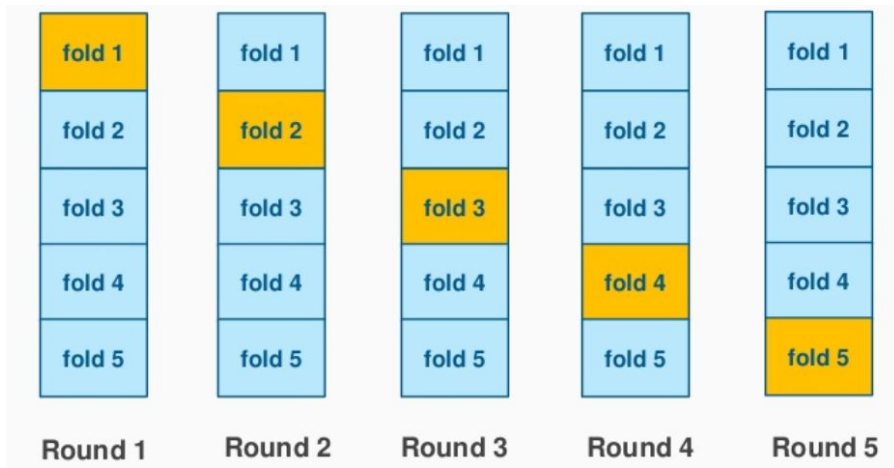
Stratified k-Fold Cross-Validation

Let's say...

- Dataset of 10 samples:
1, 2, 3, 4, 5, 6, 7, 8, 9, 10
- Labeled as Class 😊 & Class 😈
- **Stratified** 5-Fold Cross-Validation

What happens:

- Each fold maintains the original class distribution
- The model learns **from both classes** in every iteration
- **Better for imbalanced dataset**
- Prevents some folds from being dominated by a single class



Validation Set

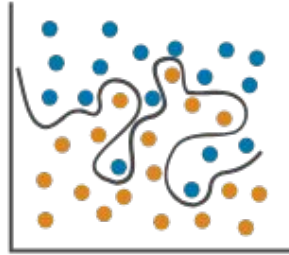
- Not the same as the test set
- Serves related purposes in model evaluation

Validation Set	Test Set
● Used during training to tune hyperparameters ● Helps detect overfitting & select the best model ● The model is evaluated on this set multiple times during training	● Used only once after training is complete ● Provides an unbiased estimate of model performance on unseen data ● Ensures the model generalizes well

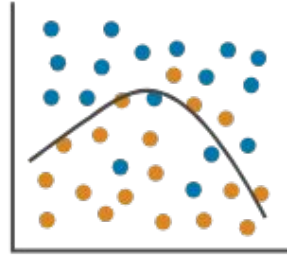
The other extreme...

Classification

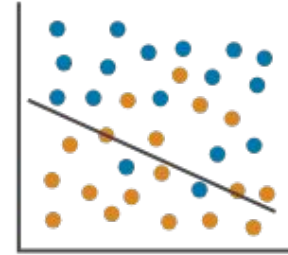
Overfitting



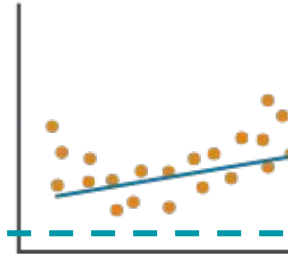
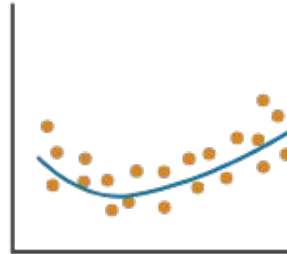
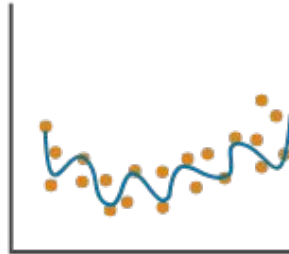
Right Fit



Underfitting



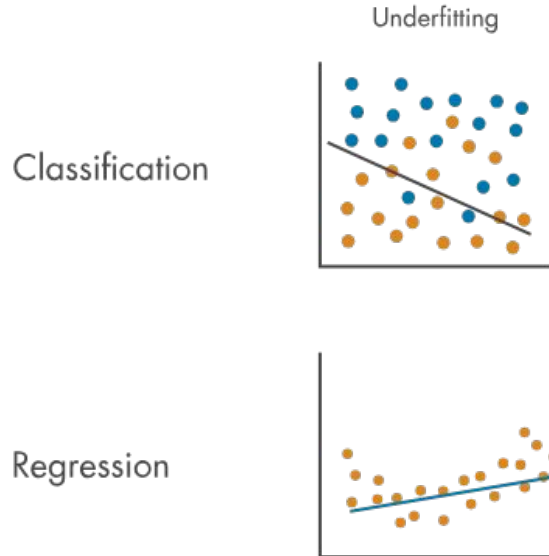
Regression



Ref: <https://www.mathworks.com/discovery/overfitting.html>

Underfitting

- Occurs when a model is too simple to capture the pattern in the data.
- Leading to poor performance on both training and test sets.



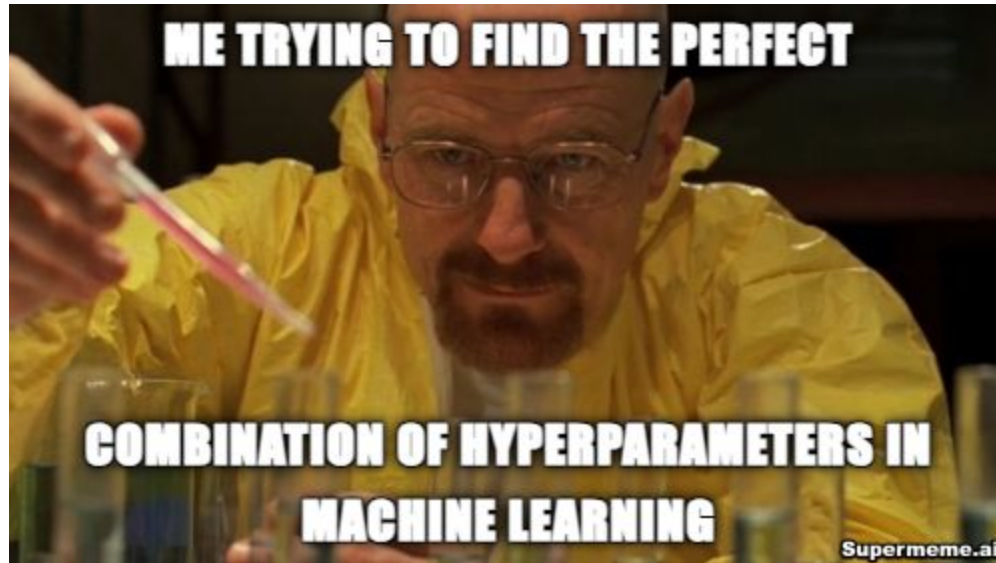
Characteristics:

- Low training and test accuracy
- Fails to learn patterns from the data
- Fails to generalize to new, unseen data
- Often caused by overly simple models

Underfitting: Prevention

- Increase Model Complexity
 - Use a more complex model
- Feature Engineering
 - Add more relevant features or transformations to improve model expressiveness.
- Reduce Regularization
 - Overuse of L1/L2 regularization can overly constrain the model.
 - Could prevent the model from learning useful patterns.
- Train Longer
- More Data
 - A small dataset may not provide enough information for learning.

Hyperparameter: How to Tune?



Ref: <https://medium.com/@deniz.kenan.kilic/changing-hyperparameters-in-machine-learning-models-101-d969f51fe414>

Hyperparameter Tuning: Manual

- Based on domain knowledge and experimentation
- Time-consuming and not efficient (like Monkey Testing)



Ref: <https://medium.com/@madangandharv2015/monkey-testing-71cbc0ddaf0>

Hyperparameter Tuning: Grid Search

Exhaustively searches the entire hyperparameter space

Decision Tree

```
max_depth  
min_samples_split  
min_samples_leaf  
max_features  
criterion  
splitter
```

Hyperparameters	Values
max_depth	{2, 3, 4}
min_samples_split	{2, 3, 4, 5}
min_samples_leaf	{1, 2, 3, 4, 5}
max_features	{3, 5, 7, 9, 11}
criterion	{Entropy, Gini}
splitter	{Best, Random}

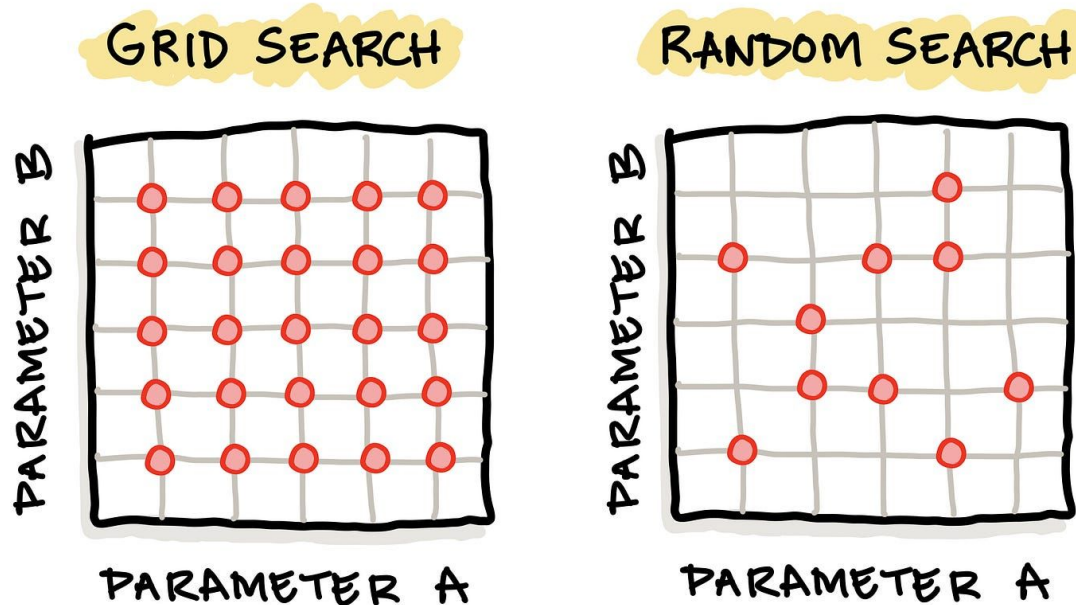
How many combinations?



Ref: <https://clear.ml/blog/how-to-do-hyperparameter-optimization-better>

Hyperparameter Tuning: Random Search

Randomly samples hyperparameter values from specified distributions



Hyperparameter Tuning: Automated

- scikit-optimize



- Hyperopt

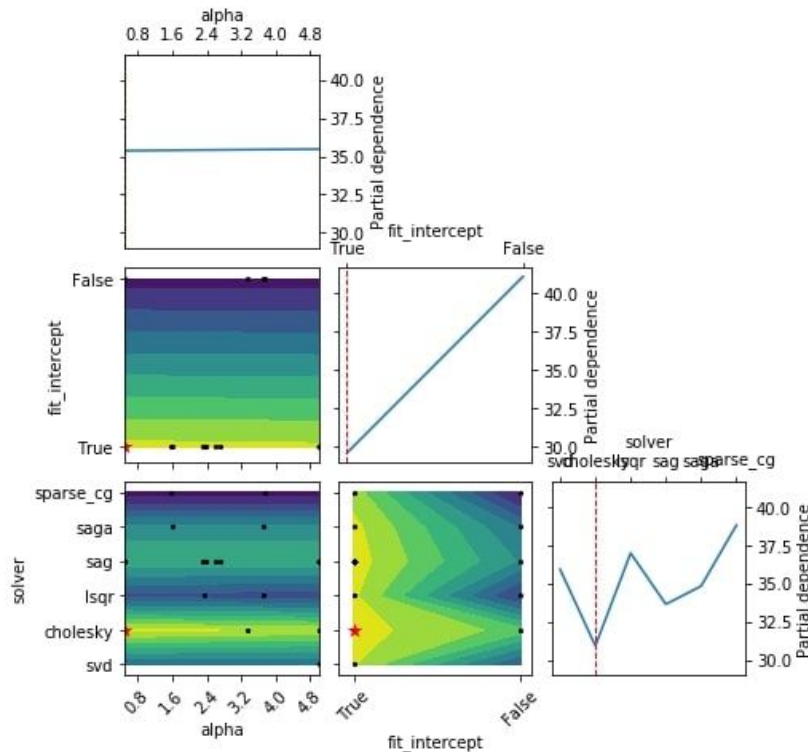


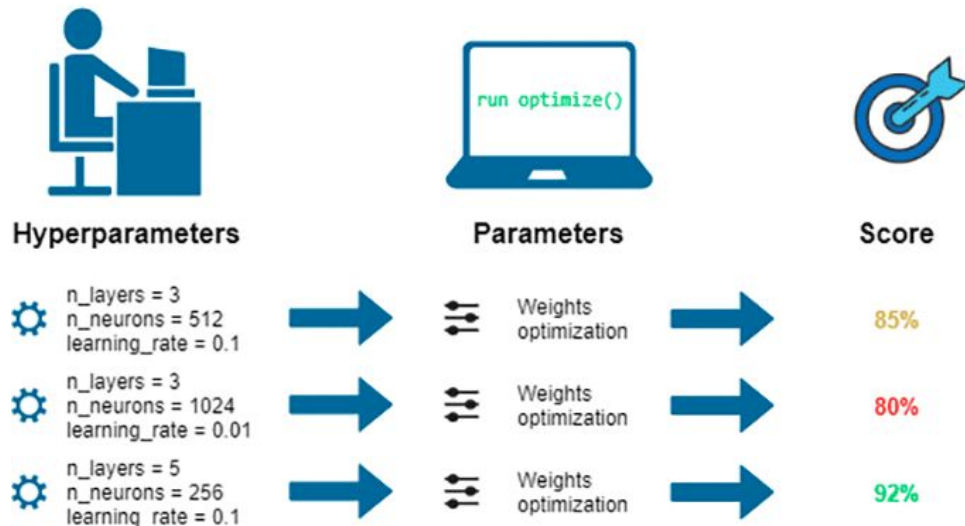
HYPEROPT

- Optuna



O P T U N A



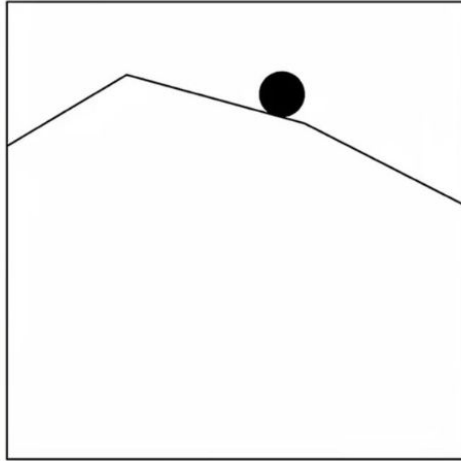


Ref: <https://coderzcolumn.com/tutorials/machine-learning/scikit-optimize-guide-to-hyperparameters-optimization>

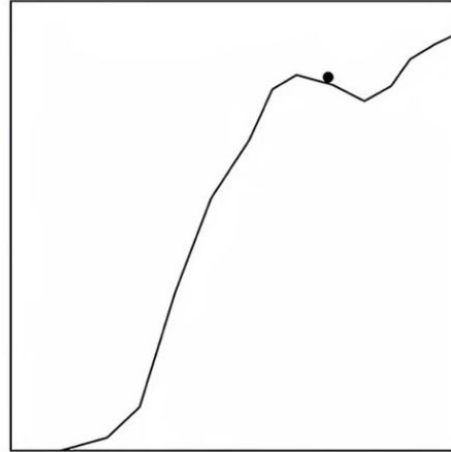
↻ You reposted



Rage *  @ragecvlt · Mar 13



WHEN IN DOUBT



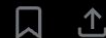
JUST ZOOM OUT

💬 34

↻ 4.7K

❤️ 29K

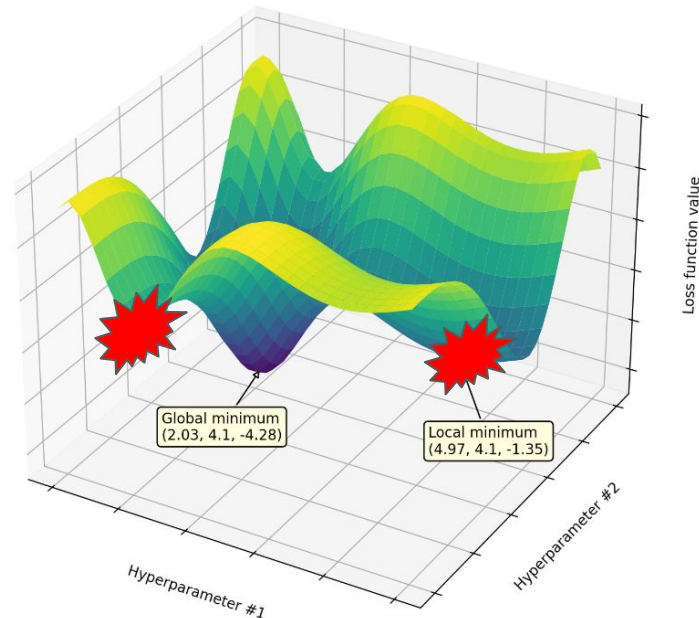
📊 490K



Ref: <https://x.com/ragecvlt/status/2032376142691172406?s=20>

Challenges in Hyperparameter Tuning

- Hyperparameter search space
- Computational resources
- Overfitting (*Oops..*)
- Noisy or incomplete information could lead to suboptimal settings



Ref:
<https://towardsdatascience.com/binary-classification-xgboost-hyperparameter-tuning-scenarios-by-non-exhaustive-grid-search-and-c261f4ce098d>

Local Minima

- Occurs when an optimization algorithm stops at a point where the loss is lower than nearby points but not the global minimum
- Usually gradient descent...

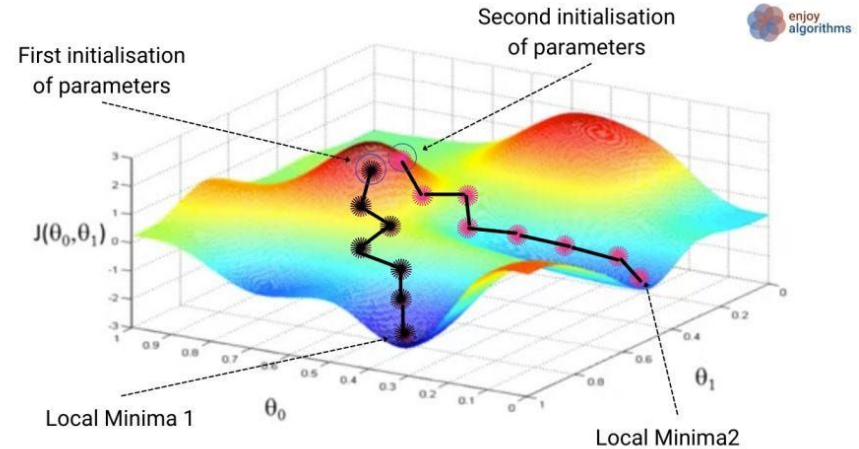


Ref: https://www.reddit.com/r/ProgrammerHumor/comments/eez8al/gradient_descent/

Local Minimum: Prevention / Mitigation

Some techniques include:

- Better initialization of parameters
- Stochastic Gradient Descent (SGD)
- Learning rate scheduling
- Adaptive optimizers
- Add noise during training
- Use better models



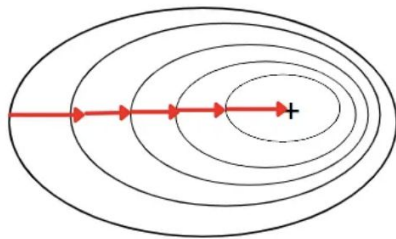
Ref: <https://www.enjoyalgorithms.com/blog/parameter-learning-and-gradient-descent-in-ml>

Comparing Gradient Descent Methods

Standard GD

- Aka Batch Gradient Descent
- Parameters are updated using the gradient computed from the *entire* dataset
- Slow!
- Computationally expensive

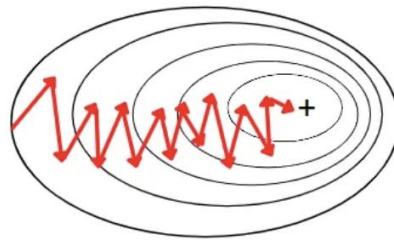
Batch Gradient Descent



Stochastic GD

- Stochastic means Random
- Updates parameters using only one sample at a time
- Fast!
- Adds noise to the optimization process

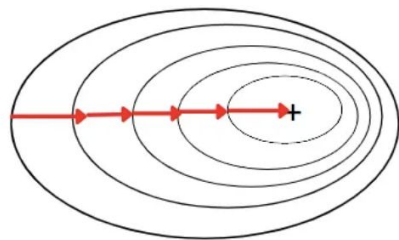
Stochastic Gradient Descent



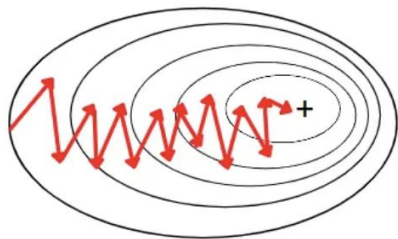
Figures from:
<https://medium.com/@waghakash0002/what-is-mean-by-gradient-descent-and-types-of-gradient-descent-d6314fd24d11>

Step	Standard GD	Stochastic GD
1	Initialize model parameters	Initialize model parameters
2	Use all training examples	Pick one training example
3	Compute predictions for entire dataset	Compute prediction for one example
4	Compute loss for entire dataset	Compute loss for one example
5	Compute gradient using all samples	Compute gradient using one sample
6	Update parameters once	Update parameters immediately
Update per epoch	1 update	n updates (n = number of samples)

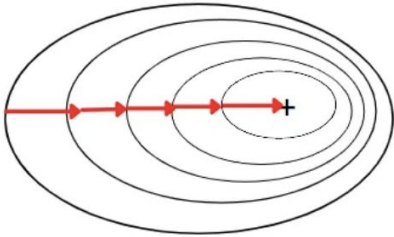
Batch Gradient Descent



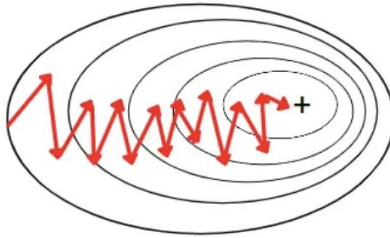
Stochastic Gradient Descent



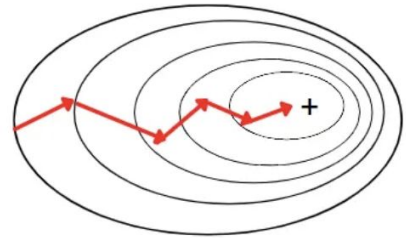
Batch Gradient Descent



Stochastic Gradient Descent



Mini-Batch Gradient Descent

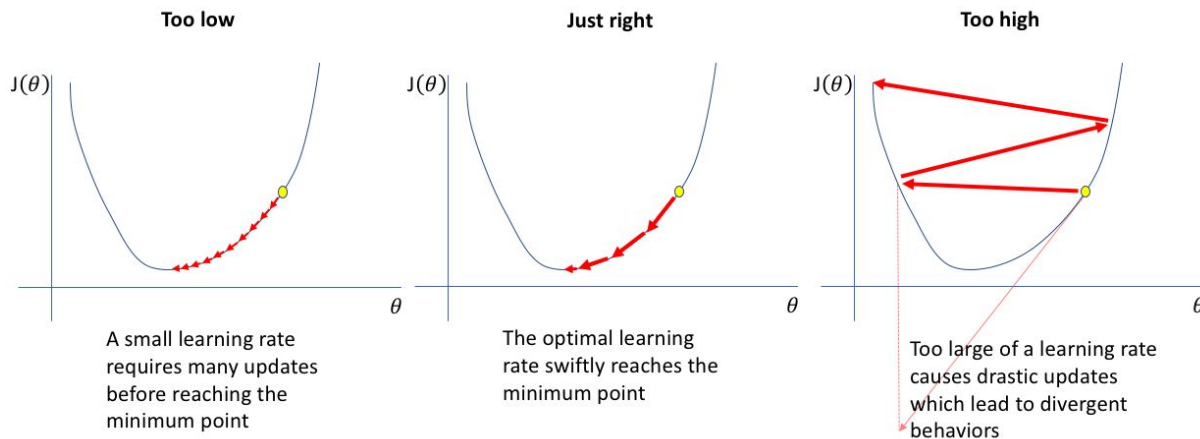


Figures from:
<https://medium.com/@waghakash0002/what-is-mean-by-gradient-descent-and-types-of-gradient-descent-d6314fd24d11>

Step	Standard GD	Stochastic GD	Mini-batch GD
1	Initialize model parameters	Initialize model parameters	Initialize model parameters
2	Use all training examples	Pick one training example	Pick a small batch (e.g., 32 or 64 samples)
3	Compute predictions for entire dataset	Compute prediction for one example	Compute predictions for batch
4	Compute loss for entire dataset	Compute loss for one example	Compute loss for batch
5	Compute gradient using all samples	Compute gradient using one sample	Compute gradient using batch
6	Update parameters once	Update parameters immediately	Update parameters after each batch
Update per epoch	1 update	n updates (n = number of samples)	n / batch_size updates

Learning Rate Scheduling

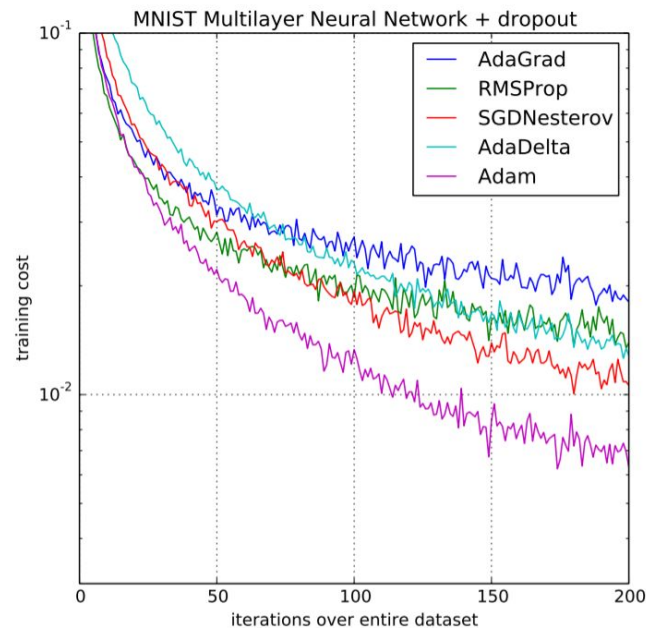
- A high learning rate early helps *jump out* of local minima
- Then it is reduced later for stable convergence



Ref: <https://www.jeremyjordan.me/nn-learning-rate/>

Adaptive Optimizers

- Modern optimizers *adjust learning rates automatically*
- Adjust step sizes per parameter
- Can escape flat or problematic regions more easily
- Examples:
 - [Adam](#)
 - [RMSProp](#)
 - [Adagrad](#)



Ref: <https://www.geeksforgeeks.org/deep-learning/adam-optimizer/>

Evaluation

Accuracy (Acc)

- An overall correctness of the model across all classes
- A ratio of correctly predicted instances to the total number of instances.

Recall (R)

- How many of the actual positive instances were correctly predicted *by the model*.

Precision (P)

- How many of the predicted positive instances *are actually positive*.

Confusion Matrix

Actual Class	Predicted Class	
	Yes	No
	Yes	No
	TP	FN (Type II error)
	FP (Type I error)	TN

$$Acc = \frac{TP+TN}{TP+TN+FP+FN}$$

$$R = \frac{TP}{TP+FN}$$

$$P = \frac{TP}{TP+FP}$$

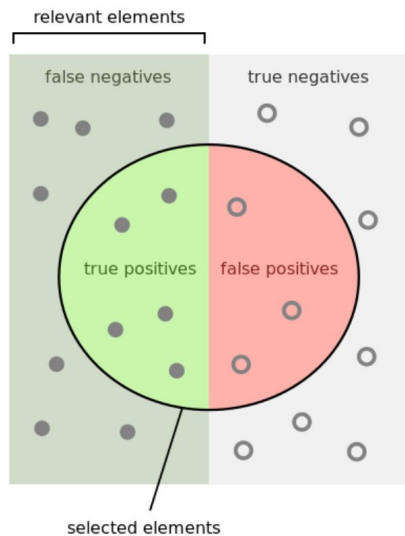
$$F1 = 2 \frac{P \times R}{P + R}$$

Accuracy (Acc) = An overall correctness of the model across all classes

Recall (R) = How many of the actual positive instances were correctly predicted by the model.

Precision (P) = How many of the predicted positive instances are actually positive.

F-measure = The harmonic mean of precision and recall



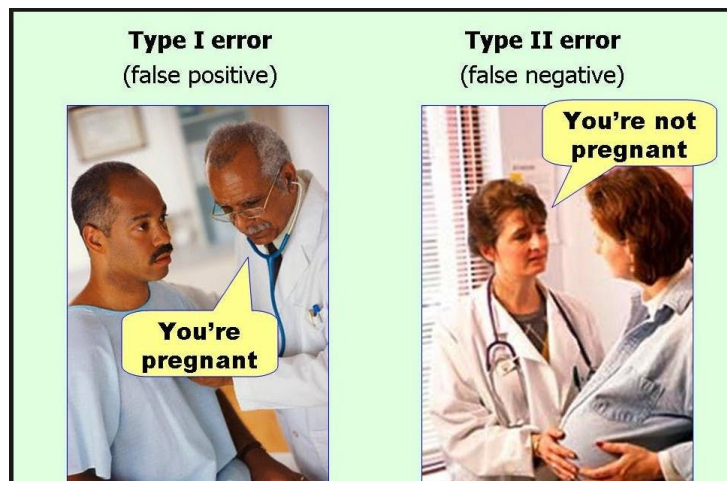
How many selected items are relevant?

$$\text{Precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

How many relevant items are selected?

$$\text{Recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

Actual Class	Predicted Class	
	Yes	No
Yes	TP	FN (Type II error)
No	FP (Type I error)	TN



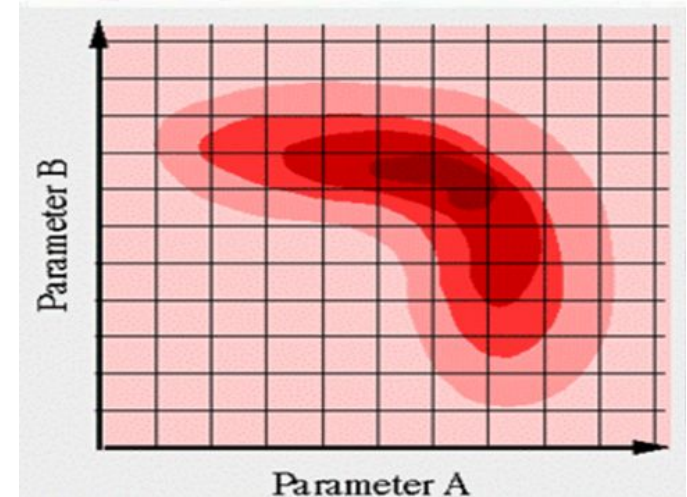
Ref: https://en.wikipedia.org/wiki/Precision_and_recall

Ref: <https://hackernoon.com/idiots-guide-to-precision-recall-and-confusion-matrix-b32d36463556>

In Summary

- Introducing model parameters
- Learnable parameters vs Hyperparameters
- Examples in ML algorithms
- Necessity & Benefits
- Important Terms
- Methods of hyperparameter tuning
- Challenges
- Evaluation

Friendships can end.
Girlfriends/boyfriends can end.
Only Hyperparameter Tuning has no **end**.



Ref: <https://www.facebook.com/share/p/18osJsaEnu/>

Q & A